

Name:

Register Number:

Course Code: ITA04

Course Name: Statistics with R Programming

Faculty Name:

1. Create numeric, character, and logical vectors and display type and content.

Aim:

To create numeric, character, and logical vectors and display type and content.

Procedure:

Step 1: Open RStudio or R console on your system.

Step 2: Create a numeric vector containing numerical values.

Step 3: Display the contents of the numeric vector on the screen.

Step 4: Use the class function to check and print the data type of the numeric vector.

Step 5: Create a character vector containing names or text values.

Step 6: Display the contents of the character vector.

Step 7: Use the class function to identify and print the data type of the character vector.

Step 8: Create a logical vector containing TRUE and FALSE values.

Step 9: Display the logical vector on the screen.

Step 10: Use the class function to print the data type of the logical vector.

Step 11: Observe and note that numeric, character, and logical vectors each have distinct data types.

```
%%R  
a=c(1,2,3,4,5,6)  
a  
print(paste("Type: ",class(a)))  
b=c("Gokul","Raghu1","Dhanush")
```

```
b
print(paste("Type: ",class(b)))
c=c(TRUE,FALSE,TRUE,FALSE)
c
print(paste("Type: ",class(c)))
```

```
[1] "Type:  numeric"
[1] "Type:  character"
[1] "Type:  logical"
```

Result:

Hence the output is verified successfully.

2. Create labeled matrices (5×4, 3×3, 2×2) filled by row/column.

Aim:

To create labeled matrices (5×4, 3×3, 2×2) filled by row/column.

Procedure:

Step 1: Open RStudio or R console on your system.

Step 2: Create a matrix by arranging a sequence of numbers into rows and columns.

Step 3: Display the created matrix to check its elements and layout.

Step 4: Create another matrix with a different size (for example, 3×3) to understand matrix creation with various dimensions.

Step 5: Display the second matrix to verify its arrangement.

Step 6: Create a smaller matrix (for example, 2×2) to observe how matrix size can vary.

Step 7: Display the third matrix to view its content.

Step 8: Observe that each matrix organizes elements in a rectangular form with equal-length rows and columns.

```
%%R
a=matrix(1:20,nrow=5,ncol=4)
a
b=matrix(1:9,nrow=3,ncol=3)
b
c=matrix(1:4,nrow=2,ncol=2)
c
```

```

      [,1] [,2]
[1,]    1    3
[2,]    2    4

```

Result:

Hence the output is verified Successfully.

3. Write an R program to create and display a 3D array with specified rows, columns, and tables.

Aim:

To write an R program to create and display a 3D array with specified rows, columns, and tables.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Create a sequence of numbers to be stored in a variable.

Step 3: Use that sequence to create a multi-dimensional array by specifying dimensions such as number of rows, columns, and tables.

Step 4: Store the created array in a variable.

Step 5: Display the array to view all the elements arranged across multiple rows, columns, and layers.

Step 6: Observe how an array in R can store data in more than two dimensions, unlike a matrix.

```

%%R
a=1:24
arr=array(a, dim=c(2,3,4))
arr

```

```

, , 1

```

```

      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

```

```

, , 2

```

```

      [,1] [,2] [,3]
[1,]    7    9   11

```

```

[2,]    8   10   12

, , 3
      [,1] [,2] [,3]
[1,]   13   15   17
[2,]   14   16   18

, , 4
      [,1] [,2] [,3]
[1,]   19   21   23
[2,]   20   22   24

```

Result:

Hence the output is verified successfully.

4. Create arrays from vectors with dimension names, print specific elements.

Aim:

To create arrays from vectors with dimension names, print specific elements.

Procedure:

Step 1: Open RStudio or R console on your system.

Step 2: Create a sequence of numbers and store it in a variable.

Step 3: Use the array function to arrange these numbers into multiple dimensions by specifying the number of rows, columns, and tables.

Step 4: Store the created array in a variable.

Step 5: Display the entire array to view how the elements are arranged in each layer.

Step 6: Access a specific element from the array using its row, column, and table position.

Step 7: Print the selected element to verify correct indexing within the array.

```

%%R
a=1:8
b=array(a, dim=c(2,2,2))
b
print(b[2,1,2])

```

```

[1] 6

```

Result:

Hence the output is verified successfully.

5. Create and manipulate factor variables (e.g., women's dataset heights, random LETTERS sample).

Aim:

Create and manipulate factor variables (e.g., women's dataset heights, random LETTERS sample).

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Create a character vector containing categorical values such as height levels.

Step 3: Convert the character vector into a factor to represent it as categorical data.

Step 4: Display the factor to observe that R assigns internal numeric codes to each category.

Step 5: Set a random seed value to ensure that the random sampling results remain the same each time you run the program.

Step 6: Randomly generate a sample of letters from A to E using the sample function.

Step 7: Convert the generated letters into a factor to represent them as categorical variables.

Step 8: Print the factor to view the sampled categories and their assigned levels.

```
##R
a = c("Short", "Medium", "Tall", "Medium", "Tall", "Short")
b = factor(a)
print(b)
set.seed(10)
c = sample(LETTERS[1:5], 8, replace = TRUE)
d = factor(c)
print(d)
```

```
[1] Short Medium Tall Medium Tall Short
Levels: Medium Short Tall
[1] C A B D C B B B
Levels: A B C D
```

Result:

Hence the output is verified successfully.

6. Create an R list containing vectors, matrices, and functions; display contents.

Aim:

To create an R list containing vectors, matrices, and functions; display contents.

Procedure

Step 1: Open RStudio or R console on your system.

Step 2: Create a numeric vector and store it in a variable.

Step 3: Create a matrix by arranging numbers into specified rows and columns.

Step 4: Define a simple function that performs a mathematical operation (for example, squaring a number).

Step 5: Combine the vector, matrix, and function into a single list using the list function.

Step 6: Assign appropriate names to each component in the list for better readability.

Step 7: Display the list to view all its components — the vector, matrix, and function — stored together in one structured object.

```
##R
a = c(1, 2, 3, 4)
b = matrix(1:6, nrow = 2, ncol = 3)
c = function(x) x^2
mylist = list(Vector = a, Matrix = b, Function = c)
print(mylist)
```

```
$Vector
[1] 1 2 3 4

$Matrix
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

$Function
function (x)
x^2
```

Result:

Hence the output is verified successfully.

7. Write R programs for basic tasks: Factors of a number, generate a vector of 10 random integers between -50 and 50, print numbers 1–100 with FizzBuzz logic.

Aim:

To write R programs for basic tasks: Factors of a number, generate a vector of 10 random integers between -50 and 50, print numbers 1–100 with FizzBuzz logic.

Procedure

Step 1: Open RStudio or R console on your system.

Step 2: Assign an integer value to a variable to represent the number for which factors are to be found.

Step 3: Initialize an empty integer vector to store the factors.

Step 4: Use a for loop to iterate through all numbers from 1 to the given number.

Step 5: Inside the loop, use the modulus operator to check if the number divides evenly (remainder equals 0).

Step 6: If the condition is true, append that value to the factors vector.

Step 7: After the loop completes, print all the factors of the given number.

Step 8: Set a random seed to ensure the random number generation gives reproducible results.

Step 9: Generate and print 10 random integers within a specified range (–50 to 50).

Step 10: Use another for loop to iterate from 1 to 100 and apply the FizzBuzz logic:

Print "FizzBuzz" for numbers divisible by both 3 and 5.

Print "Fizz" for numbers divisible only by 3.

Print "Buzz" for numbers divisible only by 5.

Otherwise, print the number itself.

Step 11: Display the complete sequence of printed outputs to verify the logic.

```
##R
n <- 12L
factors <- integer(0)
for (i in seq_len(n)) {
  if (n %% i == 0L) {
```



```

    factors <- c(factors, i)
  }
}
print("Factors of the number:")
print(factors)
set.seed(123)
print("10 random integers:")
print(sample(-50:50, 10, replace = TRUE))
for (i in 1:100) {
  if (i %% 15 == 0) print("FizzBuzz")
  else if (i %% 3 == 0) print("Fizz")
  else if (i %% 5 == 0) print("Buzz")
  else print(i)
}

```

```

[1] "Factors of the number:"
[1] 1 2 3 4 6 12
[1] "10 random integers:"
[1] -20 28 0 -37 16 -9 -1 -8 50 -37
[1] 1
[1] 2
[1] "Fizz"
[1] 4
[1] "Buzz"
[1] "Fizz"
[1] 7
[1] 8
[1] "Fizz"
[1] "Buzz"
[1] 11
[1] "Fizz"
[1] 13
[1] 14
[1] "FizzBuzz"
[1] 16
[1] 17
[1] "Fizz"
[1] 19
[1] "Buzz"
[1] "Fizz"
[1] 22
[1] 23
[1] "Fizz"
[1] "Buzz"
[1] 26
[1] "Fizz"
[1] 28
[1] 29
[1] "FizzBuzz"
[1] 31
[1] 32
[1] "Fizz"
[1] 34
[1] "Buzz"
[1] "Fizz"

```

```
[1] 37
[1] 38
[1] "Fizz"
[1] "Buzz"
[1] 41
[1] "Fizz"
[1] 43
[1] 44
[1] "FizzBuzz"
[1] 46
[1] 47
[1] "Fizz"
[1] 49
[1] "Buzz"
[1] "Fizz"
[1] 52
[1] 53
[1] "Fizz"
```

Result:

Hence the output is verified successfully.

8. Generate random numbers from a normal distribution; count occurrences.**Aim:**

To generate random numbers from a normal distribution; count occurrences.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Set a random seed value to make the random results reproducible every time you run the program.

Step 3: Generate a set of random numbers from a normal distribution using the `rnorm` function.

Step 4: Round the generated random numbers to the nearest whole number to group similar values.

Step 5: Use the `table` function to count the frequency of each rounded value.

Step 6: Convert the frequency table into a data frame for easier viewing and structured display.

Step 7: Print the data frame to observe each rounded number along with its frequency count.

```
%%R
set.seed(123)
x <- rnorm(100)
y<-(table(round(x)))
print(as.data.frame(y))
```

	Var1	Freq
1	-2	4
2	-1	21
3	0	45
4	1	22
5	2	8

Result:

Hence the output is verified successfully.

```
%load_ext rpy2.ipython
```

The rpy2.ipython extension is already loaded. To reload it, use:

```
%reload_ext rpy2.ipython
```

9. Create empty plots with specified axis limits.

Aim:

To create empty plots with specified axis limits.

Procedure

Step 1: Open RStudio or R console on your computer.

Step 2: Remove the variable named c from the workspace using the remove function.

Step 3: Use the plot function to create an empty graph by setting the type as "n".

Step 4: Define the range of x-axis and y-axis using xlim and ylim.

Step 5: Label the x-axis and y-axis with appropriate names.

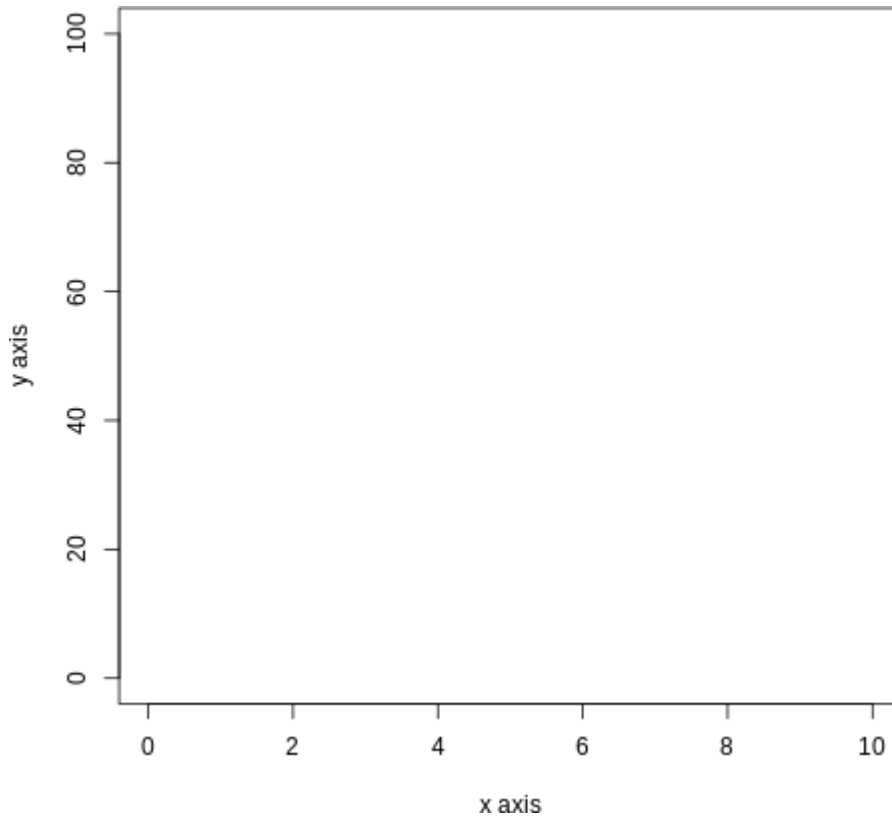
Step 6: Run the program to display an empty plotting area ready for adding points or lines later.

```
%%R
rm(c)

plot(1, type = "n",
     xlim = c(0,10),
     ylim = c(0,100),
```

```
xlab = "x axis",  
ylab = "y axis")
```

In addition: Warning message:
In rm(c) : object 'c' not found



Result:

Hence the output is verified successfully.

10. Create and explore a data frame `exam_data` with name, score, attempts, and qualifying fields. Perform extract, add row/column, sort, save to file.

Aim:

To create and explore a data frame `exam_data` with name, score, attempts, and qualifying fields. Perform extract, add row/column, sort, save to file.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Create a data frame with student details such as name, score, attempts, and qualification status.

Step 3: Add a new student record to the existing data frame using the rbind function.

Step 4: Add a new column named grade to assign grades for each student.

Step 5: Arrange the data in descending order based on the score column.

Step 6: Save the final data frame as a CSV file using the write.csv function.

Step 7: Print the data frame to display the updated and sorted student information.

```
##R
x <- data.frame(
  name = c("Anu", "Bala", "Charan", "Divya", "Esha"),
  score = c(85, 62, 90, 75, 88),
  attempts = c(1, 3, 2, 2, 1),
  qualify = c("yes", "no", "yes", "no", "yes")
)

x <- rbind(x, data.frame(name="Farhan", score=70, attempts=3, qualify="no"))
x$grade <- c("A", "C", "A+", "B", "A", "B")
x <- x[order(-x$score),]
write.csv(x, "x.csv", row.names=FALSE)
print(x)
```

	name	score	attempts	qualify	grade
3	Charan	90	2	yes	A+
5	Esha	88	1	yes	A
1	Anu	85	1	yes	A
4	Divya	75	2	no	B
6	Farhan	70	3	no	B
2	Bala	62	3	no	C

Result:

Hence the output is verified successfully.

11. Write an R program to read a .csv file and display contents.

Aim:

To write an R program to read a .csv file and display contents.

Procedure

Step 1: Open RStudio or R console on your computer.

Step 2: Use the read.csv function to load data from a CSV file into R.

Step 3: Store the imported data in a variable for further use.

Step 4: Display a message indicating that the file contents are being printed.

Step 5: Print the data to view all the rows and columns from the CSV file.

```
##R
data <- read.csv("students_copy.csv")
print("Contents of the CSV file:")
print(data)
```

```
[1] "Contents of the CSV file:"
      x
1 10
2 20
3 30
4 50
```

Result:

Hence the output is verified successfully.

12. Perform data reshaping on air quality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.

Aim:

To perform data reshaping on air quality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.

Procedure

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset `airquality` available in R.

Step 3: Use the aggregate function to calculate the mean of multiple columns (Ozone, Solar.R, Wind, and Temp) grouped by Month.

Step 4: Include the parameter `na.rm = TRUE` to ignore any missing (NA) values while calculating averages.

Step 5: Store the calculated monthly average values in a new variable.

Step 6: Print a message to indicate that monthly averages are being displayed.

Step 7: Display the result to view the average Ozone, Solar Radiation, Wind, and Temperature for each month.

```
%%R
data("airquality")
result <- aggregate(cbind(Ozone, Solar.R, Wind, Temp) ~ Month,
                    data = airquality,
                    FUN = mean,
                    na.rm = TRUE)
print("Monthly average values:")
print(result)
```

```
[1] "Monthly average values:"
  Month      Ozone  Solar.R      Wind      Temp
1     5 24.12500 182.0417 11.504167 66.45833
2     6 29.44444 184.2222 12.177778 78.22222
3     7 59.11538 216.4231  8.523077 83.88462
4     8 60.00000 173.0870  8.860870 83.69565
5     9 31.44828 168.2069 10.075862 76.89655
```

Result:

Hence the output is verified successfully.

13. Combine multiple arrays row-wise.

Aim:

To combine multiple arrays row-wise.

Procedure

Step 1: Open RStudio or R console on your system.

Step 2: Remove all existing objects from the workspace using the rm function to start fresh.

Step 3: Create three separate numeric vectors with values.

Step 4: Combine all the vectors row-wise using the rbind function to form a matrix.

Step 5: Print a message indicating that the arrays are combined row-wise.

Step 6: Display the resulting matrix to view the combined data.

```
%%R
rm(list = ls())
a <- c(1, 2, 3)
b <- c(4, 5, 6)
c <- c(7, 8, 9)
d <- rbind(a, b, c)
```

```
print("Combined arrays (row-wise):")
print(d)
```

```
[1] "Combined arrays (row-wise):"
[,1] [,2] [,3]
a     1     2     3
b     4     5     6
c     7     8     9
```

Result:

Hence the output is verified successfully.

14. Explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet).

Aim:

To explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet).

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset ChickWeight available in R.

Step 3: Display the first few rows of the dataset using the head function to understand its structure.

Step 4: Sort the dataset in ascending order based on the weight column using the order function.

Step 5: Print the sorted dataset to view the records arranged by weight.

Step 6: Use the aggregate function to calculate the average weight of chicks for each Diet group.

Step 7: Display the result showing the mean weight corresponding to each diet type.

```
%%R
data("ChickWeight")
head(ChickWeight)
sorted <- ChickWeight[order(ChickWeight$weight), ]
print(sorted)
aggregate(weight ~ Diet, ChickWeight, mean)
```

```
      weight Time Chick Diet
196       35    2    18    1
26        39    2     3    1
```


195	39	0	18	1
293	39	0	27	2
305	39	0	28	2
317	39	0	29	2
365	39	0	33	3
401	39	0	36	3
543	39	0	48	4
13	40	0	2	1
221	40	0	21	2
269	40	0	25	2
519	40	0	46	4
555	40	0	49	4
49	41	0	5	1
61	41	0	6	1
73	41	0	7	1
108	41	0	10	1
132	41	0	12	1
144	41	0	13	1
156	41	0	14	1
168	41	0	15	1
176	41	0	16	1
209	41	0	20	1
233	41	0	22	2
353	41	0	32	3
377	41	0	34	3
389	41	0	35	3
413	41	0	37	3
425	41	0	38	3
449	41	0	40	3
507	41	0	45	4
531	41	0	47	4
567	41	0	50	4
1	42	0	1	1
37	42	0	4	1
50	42	2	5	1
85	42	0	8	1
96	42	0	9	1
183	42	0	17	1
257	42	0	24	2
281	42	0	26	2
329	42	0	30	2
341	42	0	31	3
437	42	0	39	3
461	42	0	41	4
473	42	0	42	4
485	42	0	43	4
497	42	0	44	4
25	43	0	3	1
120	43	0	11	1
197	43	0	19	1
245	43	0	23	2
109	44	2	10	1
177	45	2	16	1
294	46	2	27	2
306	46	2	28	2

Result:

Hence the output is verified successfully.

15. Perform EDA on iris dataset: dimensions, summary, standard deviation, quantiles, grouping by Species, pivot table, categorical grouping with Sepal.Length categories.

Aim:

To perform EDA on iris dataset: dimensions, summary, standard deviation, quantiles, grouping by Species, pivot table, categorical grouping with Sepal.Length categories.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset iris available in R.

Step 3: Use the dim function to find the number of rows and columns in the dataset.

Step 4: Apply the summary function to view basic statistical details such as minimum, maximum, mean, and quartiles for each variable.

Step 5: Use the sapply function with sd to calculate the standard deviation of the numeric columns.

Step 6: Find the quantile values of the Sepal.Length column to understand data distribution.

Step 7: Use the aggregate function to calculate the mean of all numeric variables grouped by Species.

Step 8: Display the results to observe the overall statistical analysis of the dataset.

```
%%R
data("iris")
dim(iris)
summary(iris)
sapply(iris[,1:4], sd)
quantile(iris$Sepal.Length)
aggregate(. ~ Species, iris, mean)
```

	Species	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	setosa	5.006	3.428	1.462	0.246
2	versicolor	5.936	2.770	4.260	1.326
3	virginica	6.588	2.974	5.552	2.026

Result:

Hence the output is verified successfully.

16. Explore USArrests dataset: summary statistics, state with largest rape arrests, max & min murder rates, correlation among features, states above median assault arrests and bottom 25% for murder, visualization with histogram, density, scatterplots, bar graphs.

Aim:

To explore USArrests dataset: summary statistics, state with largest rape arrests, max & min murder rates, correlation among features, states above median assault arrests and bottom 25% for murder, visualization with histogram, density, scatterplots, bar graphs.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset iris available in R.

Step 3: Use the dim function to find the number of rows and columns in the dataset.

Step 4: Apply the summary function to view basic statistical details such as minimum, maximum, mean, and quartiles for each variable.

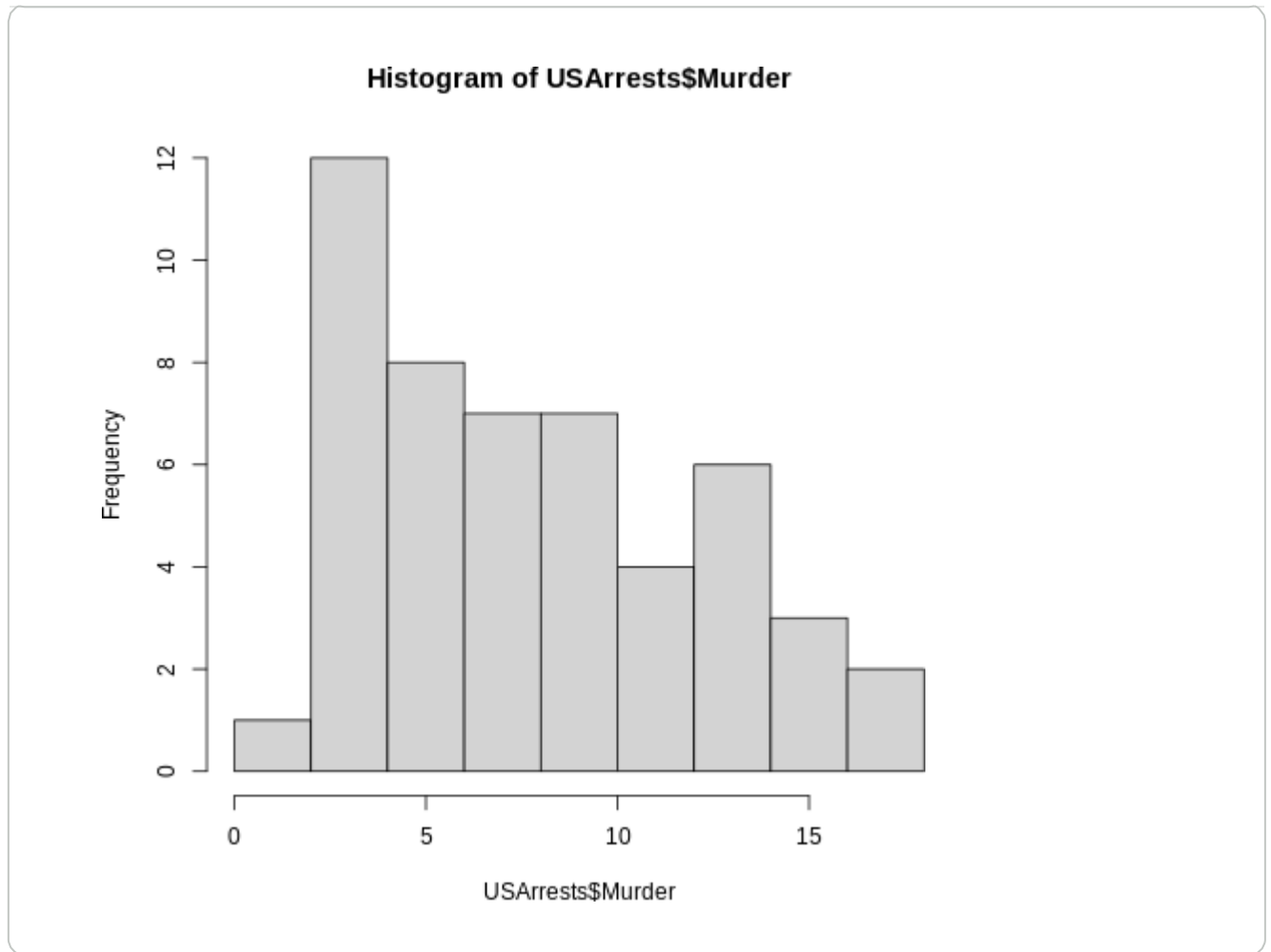
Step 5: Use the sapply function with sd to calculate the standard deviation of the numeric columns.

Step 6: Find the quantile values of the Sepal.Length column to understand data distribution.

Step 7: Use the aggregate function to calculate the mean of all numeric variables grouped by Species.

Step 8: Display the results to observe the overall statistical analysis of the dataset.

```
##R
data("USArrests")
summary(USArrests)
which.max(USArrests$Rape)
max(USArrests$Murder)
min(USArrests$Murder)
cor(USArrests)
hist(USArrests$Murder)
```

**Result:**

Hence the output is verified successfully.

17. Explore Titanic dataset: bar chart of survival vs class, modify plot by gender, histogram of Age.**Aim:**

To explore Titanic dataset: bar chart of survival vs class, modify plot by gender, histogram of Age.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset Titanic available in R.

Step 3: Convert the Titanic dataset into a data frame format for easier manipulation and plotting.

Step 4: Create a bar plot showing the relationship between Class and Survival using the table function to count frequencies.

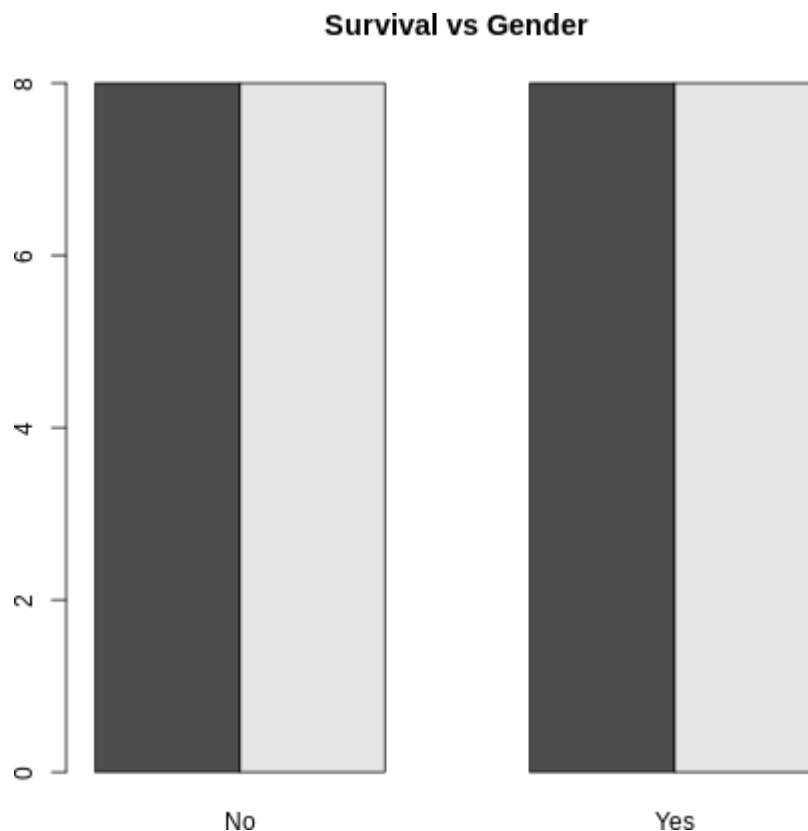
Step 5: Add a title to the plot to indicate that it represents Survival vs Class.

Step 6: Create another bar plot to show the relationship between Gender and Survival.

Step 7: Add an appropriate title for the second plot to represent Survival vs Gender.

Step 8: Observe both plots to compare survival patterns across different classes and genders.

```
##R
data("Titanic")
t <- as.data.frame(Titanic)
barplot(table(t$Class, t$Survived), main="Survival vs Class", beside=TRUE)
barplot(table(t$Sex, t$Survived), main="Survival vs Gender", beside=TRUE)
```



Result:

Hence the output is verified successfully.

18. Create graphs in R: boxplot, histogram, bar plot, line chart, scatter plot.

Aim:

To create graphs in R: boxplot, histogram, bar plot, line chart, scatter plot.

Procedure:

Step 1: Open RStudio or R console on your computer.

Step 2: Create two numeric vectors containing values for plotting.

Step 3: Use the par function with mfrow to divide the plotting area into multiple sections for displaying different charts together.

Step 4: Draw a boxplot to show the distribution of the data.

Step 5: Create a histogram to visualize the frequency of values.

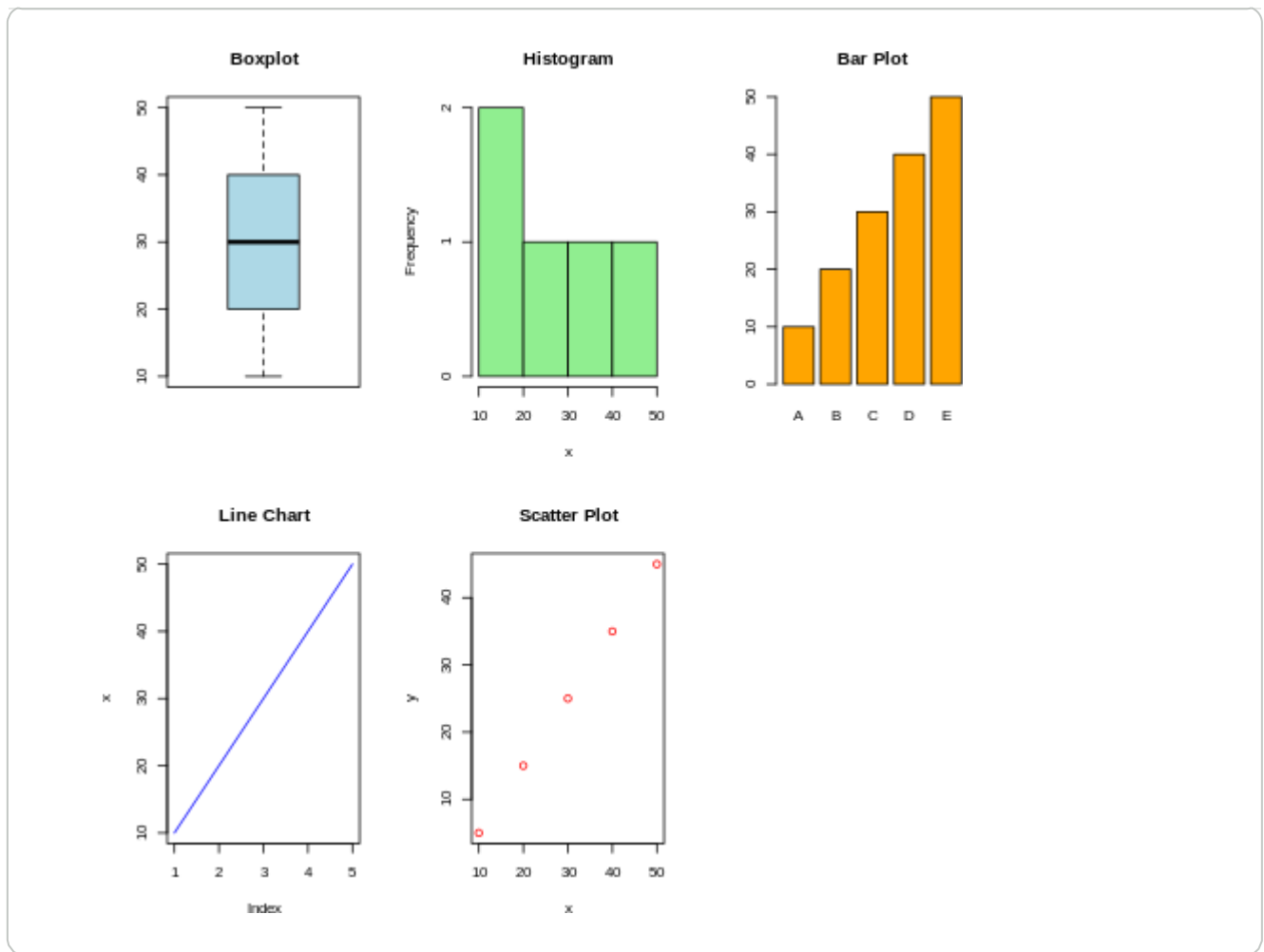
Step 6: Generate a bar plot with labeled bars to compare values across categories.

Step 7: Plot a line chart to show the trend of values sequentially.

Step 8: Draw a scatter plot to visualize the relationship between two numeric variables.

Step 9: Observe all the graphs displayed in the multi-plot layout.

```
##R
x <- c(10, 20, 30, 40, 50)
y <- c(5, 15, 25, 35, 45)
par(mfrow = c(2, 3))
boxplot(x, main="Boxplot", col="lightblue")
hist(x, main="Histogram", col="lightgreen")
barplot(x, main="Bar Plot", col="orange", names.arg=c("A","B","C","D","E"))
plot(x, type="l", main="Line Chart", col="blue")
plot(x, y, main="Scatter Plot", col="red")
```

**Result:**

Hence the output is verified successfully.

19. Build a regression model on advertising dataset ($\text{Sales} \sim \text{Spend}$) and predict Sales.

Aim:

To build a regression model on advertising dataset ($\text{Sales} \sim \text{Spend}$) and predict Sales.

Procedure

Step 1: Open RStudio or R console on your computer.

Step 2: Create two numeric vectors containing values for plofling.

Step 3: Use the par function with mfrow to divide the plofling area into multiple sections for displaying different charts together.

Step 4: Draw a boxplot to show the distribution of the data.

Step 5: Create a histogram to visualize the frequency of values.

Step 6: Generate a bar plot with labeled bars to compare values across categories.

Step 7: Plot a line chart to show the trend of values sequentially.

Step 8: Draw a scatter plot to visualize the relationship between two numeric variables.

Step 9: Observe all the graphs displayed in the multi-plot layout.

```
##R
Spend <- c(10, 20, 30, 40, 50)
Sales <- c(15, 25, 35, 45, 55)
model <- lm(Sales ~ Spend)
predict(model, data.frame(Spend = 60))
```

```
1
65
```

Result:

Hence the output is verified successfully.

20. Create multiple regression model using ChickWeight dataset with "Time" and "Diet" as predictors; predict weight and compute model error.

Aim:

To create multiple regression model using ChickWeight dataset with "Time" and "Diet" as predictors; predict weight and compute model error.

Procedure

Step 1: Open RStudio or R console on your computer.

Step 2: Load the built-in dataset ChickWeight available in R.

Step 3: Use the lm function to create a linear regression model where weight is predicted using Time and Diet as independent variables.

Step 4: Use the predict function to generate predicted weight values based on the fitted model.

Step 5: Calculate the mean squared error (MSE) by finding the average of squared differences between the actual and predicted weight values.

Step 6: Display the calculated MSE to evaluate the accuracy of the regression model.


```
%%R
data("ChickWeight")
m <- lm(weight ~ Time + Diet, data = ChickWeight)
p <- predict(m)
mean((ChickWeight$weight - p)^2)

[1] 1284.319
```

Result:

Hence the output is verified successfully.

21. Randomly split iris dataset into train/test (80/20), build logistic regression (Species ~ Petal.Length + Petal.Width), predict, and evaluate with confusion matrix.

Aim:

To randomly split iris dataset into train/test (80/20), build logistic regression (Species ~ Petal.Length + Petal.Width), predict, and evaluate with confusion matrix.

```
%%R
data("iris")
iris <- subset(iris, Species %in% c("setosa", "versicolor"))
m <- glm(Species ~ Petal.Length + Petal.Width, data=iris, family=binomial)
p <- ifelse(predict(m, type="response") > 0.5, "versicolor", "setosa")
table(p, iris$Species)
```

```
p          setosa versicolor virginica
setosa      50          0          0
versicolor  0          50          0
```

In addition: Warning messages:

- 1: glm.fit: algorithm did not converge
- 2: glm.fit: fitted probabilities numerically 0 or 1 occurred